Chest X-Ray Project

The easy guide: everything from the start

What we built, which files we added (one by one),
which models we tried (one by one), and what is next.

October 2026

What is in this guide

Part	Topic	Page
1	The big idea (in 1 page)	3
2	Words you need to know	4
3	The data (the X-ray photos)	5
4	The models, one by one	6
5	The files, one by one	9
6	The web page (Streamlit)	12
7	What we found: the big problem	13
8	What is next	14
9	Cheat sheet: commands	15

How to read this guide: read Part 1 first. If a word is new, look in Part 2. You do not need to read everything in one day.

Part 1. The big idea

The goal

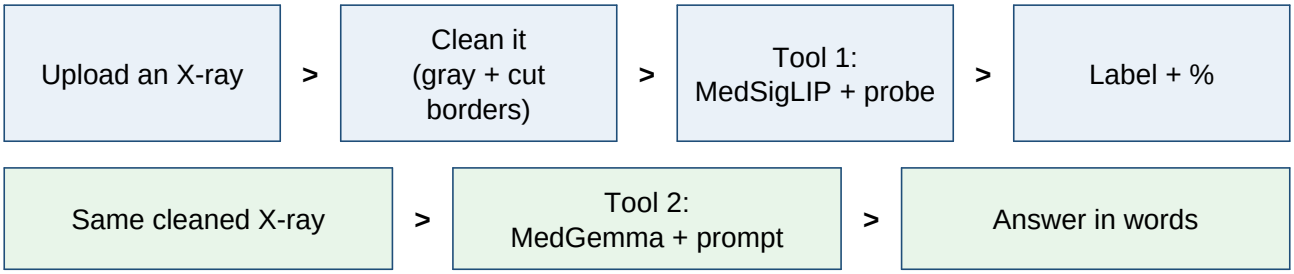
We want a computer program that looks at a **chest X-ray** and says what it sees: **Normal**, **Pneumonia**, **Covid-19** or **Tuberculosis**.

This program is a **tool**. Later, an **AI agent** will use this tool. That is the "Agentic AI" part of the project.

We have 2 tools

	Tool 1: MedSigLIP + linear probe	Tool 2: MedGemma
Who made it	Google (we trained a small part)	Google (we trained nothing)
What it gives	A label + how sure (for example "Normal 92%")	An answer in words (for example "Pneumonia")
Speed	Fast (under 1 second)	Slow (a few seconds)
How we make it better	Train the small part (the probe)	Write a better question (the prompt)

How one X-ray goes through the system



The 3 rules from the professor

- **No fine-tuning.** We do not change the big models. We only train a tiny part on top.
- **No class weights.** We use the data as it is. No special help for Covid-19.
- **Not ResNet / DenseNet.** They are too basic. Use newer, better models.

Part 2. Words you need to know

Word	Simple meaning
Model	A computer "brain" that learned from many pictures.
Pretrained model	A model that somebody (Google, Meta, Microsoft) already trained. We download it.
Frozen	We do not change the model. It stays exactly the same.
Embedding (features)	The model turns a photo into a list of numbers that describe it. MedSigLIP gives 1,152 numbers per photo.
Linear probe	A very small, simple classifier. It reads the 1,152 numbers and picks the disease. This is the ONLY part we train.
Epoch	One full pass over all training photos. We train the probe for 10 epochs.
Training / Validation / Testing	Training = photos to learn from. Validation = photos to check and choose the best version. Testing = the final exam, used once.
Accuracy	How many answers are correct. 95% = 95 of 100 correct.
Confidence	How sure the model is. 47% = not sure. 99% = very sure.
Zero-shot	Use the model with no training at all. Just ask.
Prompt	The question we write to MedGemma.
Few-shot	In the prompt, we also show MedGemma a few solved examples first.
Prompt engineering	Change the prompt step by step to get better answers.
Shortcut	The model learns an easy wrong clue (for example the hospital or the age), not the disease itself.
Streamlit	A Python tool that turns a Python file into a web page.

Part 3. The data

We got the dataset from Kaggle (a zip from the manager). It has **15,600 chest X-rays** in 4 groups. It is already split into Training, Validation and Testing.

The original folder is **Chest_X-ray** (next to the project folder). We never change it.

Step: clean the data (file: data_check.py)

- Looked for empty or broken photos: found 0.
- Looked for the same photo saved 2 times: found and removed **38**.
- Copied the good photos to **data/processed/**. All models use this copy.

How many photos

	Covid-19	Normal	Pneumonia	Tuberculosis	Total
Training	2,699	2,697	2,688	2,693	10,777
Validation	500	500	494	500	1,994
Testing	700	700	693	698	2,791

Important to remember: each disease comes from a **different source**. All Pneumonia photos are **young children** from one hospital in China. Covid-19 and Tuberculosis photos are mostly **adults** from other hospitals. This causes a big problem later (see Part 7).

Part 4. The models, one by one

We tried models in this order. Each step taught us something.

Model 1: DenseNet121 (fine-tuned)

What	Result	Status
An older image model (from 2017). We trained the whole model on our photos.	90.6% test accuracy	REJECTED. Professor: too basic, and we should not fine-tune.

Files: preprocessing.py, data_loader.py, model.py, train.py, evaluate.py. We keep them only as a record. We do not use them now.

Model 2: DINOv2 (Meta) + linear probe

What	Result	Status
A modern model trained on normal photos (cats, cars, ...). Frozen. We trained only the probe.	88.8% test accuracy	Tried. Not the best.

Model 3: RAD-DINO (Microsoft) + linear probe

What	Result	Status
Like DINOv2, but trained on chest X-rays. Frozen + probe.	87.9% test accuracy	Tried. Not the best.

Model 4: MedSigLIP (Google) + linear probe, 4 labels

What	Result	Status
A model trained on medical images AND medical text. Frozen + probe. 4 labels.	**95.6%** test accuracy	**CHOSEN.** Best model. Used on the web page now.

We also tried MedSigLIP in other ways: zero-shot (no training) 48.6%, k-NN 84.9%. The linear probe was clearly the best.

Part 4. The models (continued)

All classifier results side by side

Model	Learned from	Covid-19	Normal	Pneumonia	TB	All
DenseNet121 (fine-tuned)	normal photos	84.4%	81.4%	98.3%	98.3%	90.6%
DINOv2 + probe	normal photos	85.6%	79.6%	94.9%	95.1%	88.8%
RAD-DINO + probe	chest X-rays	84.4%	75.4%	96.7%	95.3%	87.9%
MedSigLIP + probe	medical images + text	**93.6%**	**92.1%**	**98.1%**	**98.7%**	**95.6%**

Model 5: MedSigLIP + linear probe, only 2 labels

What	Result	Status
The same MedSigLIP, but the probe learns only Normal vs Pneumonia.	98.35% test accuracy	Made and saved. NOT on the web page now (you asked to go back to 4 labels).

Model 6: MedGemma (Google) + prompt engineering

MedGemma is a model that can **look at a picture and answer in words**, like a chat. We do not train it. We only change the **prompt** (the question).

Prompt	Idea	Labels	Result
v1	Answer in one word	4	5 / 10 training photos
v2	Describe first, then answer	4	5 / 10
v3	Long step-by-step rules	4	1 / 10 (worse: long prompts confuse it)
2c_v1	Normal or Pneumonia? One word	2	Validation 32 / 40 (80%)
2c_fs1	2c_v1 + 1 solved example per label (few-shot)	2	**Validation 35 / 40 (87.5%)** - our best

Not done yet: the final **test** for MedGemma (70 Normal + 70 Pneumonia photos). So we do not have MedGemma's final accuracy yet.

Part 4. The models (summary)

#	Model	Type	Best result	Used now?
1	DenseNet121	Fine-tuned (whole model trained)	90.6%	No (rejected)
2	DINOv2	Frozen + probe	88.8%	No
3	RAD-DINO	Frozen + probe	87.9%	No
4	MedSigLIP, 4 labels	Frozen + probe	95.6%	**Yes, web page**
5	MedSigLIP, 2 labels	Frozen + probe	98.35%	Saved, not on page
6	MedGemma	No training, only prompts	87.5% (validation)	**Yes, web page**

The saved model files (folder: models/)

File	What it is
best_densenet121.pth	Model 1 (old, not used)
probe_dinov2-base.pth	Model 2 probe (not used)
probe_rad-dino.pth	Model 3 probe (not used)
probe_medsiglip-448.pth	**Model 4 probe, 4 labels. Used by the web page.**
probe_medsiglip-448_2class.pth	Model 5 probe, 2 labels
xray_averages_medsiglip-448.pt	Used by the safety checker (is this photo an X-ray?)

MedSigLIP and MedGemma themselves are **not** in this folder. They are big (GBs). They download from Hugging Face the first time and stay in a cache on your Mac.

Part 5. The files, one by one

All files are in the folder **chest-xray-pneumonia**. Here they are in the order we added them. The column **"You run it?"** tells you if you start this file yourself.

Stage A: the start (settings and data)

#	File	What it does	You run it?
1	src/config.py	All settings in one place: folders, the 4 disease names, model names, batch size, epochs, limits.	No. Other files read it.
2	src/utils.py	Small helpers: set_seed (same random numbers every time), save and load a model.	No
3	src/data_check.py	Removes broken and duplicate photos. Copies clean photos to data/processed/.	Yes, once (done)
4	notebooks/01_data_exploration.ipynb	Looks at the data: counts, example photos, sizes.	Optional

Stage B: the first model (old, not used now)

#	File	What it does	You run it?
5	src/preprocessing.py	Makes every photo 224 x 224 and turns it into numbers.	No
6	src/data_loader.py	Reads the photos from the folders in groups (batches).	No
7	src/model.py	Builds DenseNet121 / ResNet18.	No
8	src/train.py	Trains DenseNet121 (Model 1).	Not any more
9	src/evaluate.py	Tests DenseNet121 on the Testing photos.	Not any more

Part 5. The files (continued)

Stage C: the frozen models + linear probe

#	File	What it does	You run it?
10	src/dinov2_classify.py	The main classifier file. Uses DINOv2, RAD-DINO or MedSigLIP (choose in config.py: DINOV2_NAME). Turns all photos into numbers, trains the linear probe for 10 epochs, tests it, saves it. Made Models 2, 3 and 4.	Yes (done)

Stage D: MedGemma prompt engineering

#	File	What it does	You run it?
11	src/prompts.py	All the prompts (v1, v2, v3, 2c_v1, 2c_fs1), each with a note about its result. CURRENT_VERSION = the prompt to use.	No. You EDIT it.
12	src/medgemma_helper.py	The technical part: picks photos, loads MedGemma, asks it, saves the answers in a log. You do not need to change it.	No
13	src/prompt_engineering.py	The simple file. Set MODE = "try", "validate" or "test", then run. It prints MedGemma's answers and the accuracy.	**Yes**

How prompt engineering works: write a prompt in prompts.py, then run prompt_engineering.py with MODE = "try" (10 + 10 photos). Read the answers and change one thing. Repeat. When it is good, use MODE = "validate" (20 + 20). At the very end, use MODE = "test" (70 + 70) only once.

Stage E: the web page

#	File	What it does	You run it?
14	src/predict.py	One X-ray in, label + confidence out. Loads MedSigLIP and the saved probe.	No. app.py uses it.
15	src/clean_image.py	Cleans an uploaded photo: makes it gray and cuts empty black or white borders, so it looks like our dataset.	No. app.py uses it.
16	app.py	The web page (Streamlit). Upload an X-ray, see MedSigLIP's answer with 4 bars, click "Ask MedGemma".	**Yes** (see Part 6)

Part 5. The files (continued)

Stage F: 2 labels and safety checks (made, but not on the page now)

#	File	What it does	You run it?
17	src/train_2class_probe.py	Trains Model 5 (MedSigLIP, only Normal vs Pneumonia). You ran it: 98.35%.	Yes (done)
18	src/xray_check.py	Safety checker with 3 checks: (1) is it a chest X-ray? (2) is it Normal or Pneumonia, not another disease? (3) is the model sure (80% or more)? If not, it skips the photo or marks it "needs review".	No

Stage G: tests (to check the code works)

#	File	What it does	You run it?
19	tests/test_xray_rules.py	30 fast tests of the safety rules (no model needed).	Optional
20	tests/test_xray_integration.py	Slow tests with the real models and the web page.	Optional
21	tests/conftest.py, pytest.ini	Settings for the tests.	No

Other files

File	What it is
README.md	The description of the project on GitHub.
requirements.txt	The list of Python libraries the project needs.
.gitignore	Tells Git which big folders NOT to upload (data, models, outputs).

The folders

Folder	What is inside
src/	All the Python code
data/processed/	The clean copy of the photos
models/	The saved probes (see Part 4)
outputs/features/	The saved numbers (embeddings) of every photo, so we do not compute them again
outputs/prompt_logs/	Every MedGemma answer from prompt engineering
outputs/logs/	The data check report
tests/	The tests

Part 6. The web page (Streamlit)

Streamlit turns the Python file **app.py** into a web page. It runs only on your Mac.

How to start it

Open the terminal and type these 2 lines:

```
cd ~/Desktop/Alzaimer-AgenticAI/chest-xray-pneumonia
../.venv/bin/streamlit run app.py --server.port 8502
```

Then open **http://localhost:8502** in your browser. To stop it, press **Ctrl + C** in the terminal.

What you see on the page

Place	What it shows
Left sidebar	Choose the MedGemma prompt (v1, v2, v3, 2c_v1, 2c_fs1). It shows the prompt text.
Upload box	Upload a chest X-ray (png or jpg).
Left side	The cleaned X-ray. Click "Show the original upload" to see the photo before cleaning.
Right side, top	MedSigLIP: the prediction and 4 bars (Covid-19, Normal, Pneumonia, Tuberculosis).
Yellow warning	If the top answer is below 80%: "Low confidence: a doctor should check this X-ray."
Ask MedGemma button	MedGemma looks at the X-ray. "MedGemma said" = its words. "Read as" = the label we read from its words.

> The first time, the page needs about 1 minute to load MedSigLIP and MedGemma. After that it is fast.

Part 7. What we found: the big problem

We tested both tools on 2 real **pneumonia** X-rays from Google (not from our dataset).

Photo	MedSigLIP + probe said	MedGemma said	Correct answer
Adult with pneumonia	Normal (only 47% sure) + warning	**Pneumonia**	Pneumonia
Older child, standing photo	Tuberculosis (81% sure, no warning)	**Pneumonia**	Pneumonia

Why MedSigLIP was wrong

- In our dataset, **all Pneumonia photos are young children from one hospital**, lying down.
- Covid-19 and Tuberculosis photos are mostly **adults**, standing, from other hospitals.
- So the probe learned a **shortcut**: "what the photo looks like" (age, hospital, position), not "what the disease looks like".
- On our own test photos (same style) it gets 95-98%. On new photos it can fail.

Why MedGemma was right

- Google trained it on very many X-rays: adults, children, many hospitals.
- We did not train it on our dataset, so it did not learn our shortcut.

Simple picture: MedSigLIP + probe is a student who studied only one book. Very good on questions from that book, weak on new questions. MedGemma studied many books, so it does better on new questions.

Why we should NOT train MedGemma on our data: it would learn the same shortcut. Its strength is that it was NOT trained on our dataset.

Part 8. What is next

Step	What to do	Why
1	Wait for the professor's answer: "Can I use only MedGemma?"	This decides the plan.
2	Test MedGemma once on the test photos: in prompt_engineering.py set MODE = "test" and run it (about 15-30 minutes).	To get MedGemma's final accuracy. The professor will ask for it.
3	Test both tools on more outside photos (from other hospitals).	To show the shortcut problem with numbers.
4	Maybe: add pneumonia photos of adults from other datasets.	So each disease comes from many sources. Ask the professor first.
5	Build the AI agent: it uses MedSigLIP and MedGemma as tools, and asks a doctor when it is not sure.	This is the final goal of the project.

Questions for the professor

- Do you agree the problem is the dataset (shortcut), not the model?
- Can I use only MedGemma? Or both tools, and trust the answer only when they agree?
- Can I add outside photos to the training data?
- Should I report results on outside photos next to the 95.6%?

Part 9. Cheat sheet: commands

Always start in the project folder:

```
cd ~/Desktop/Alzaimer-AgenticAI/chest-xray-pneumonia
```

I want to ...	Type this
Start the web page	`../.venv/bin/streamlit run app.py --server.port 8502`
Stop the web page	Press Ctrl + C in the terminal
Run prompt engineering	`../.venv/bin/python src/prompt_engineering.py`
Predict one X-ray (no web page)	`../.venv/bin/python -m src.predict photo.png`
Train the classifier again	`../.venv/bin/python -m src.dinov2_classify`
Run the fast tests	`../.venv/bin/python -m pytest tests/test_xray_rules.py`
Log in to Hugging Face	`hf auth login` (never share your token)

Where to change things

I want to change ...	Open this file
The MedGemma prompt	src/prompts.py (write a new version, set CURRENT_VERSION)
try / validate / test	src/prompt_engineering.py (MODE = ...)
Which model the classifier uses	src/config.py (DINOV2_NAME)
The web page	app.py

Remember: this project is for research and learning. It is not a medical diagnosis.